

Annexes demandees : scenarios de demonstration et extraits de app.py

Ces deux annexes complètent explicitement les éléments demandés dans le rapport : les scénarios de démonstration pratique et les extraits du code source principal app.py.

Annexe I - Scenarios de demonstration

Les scenarios suivants peuvent etre utilises pendant la soutenance pour montrer le fonctionnement reel du site web e-learning.

No	Scenario	Etapes	Resultat attendu
1	Demonstration accueil	Ouvrir http://127.0.0.1:5000	La page d'accueil affiche la plateforme et les cours proposes.
2	Connexion etudiant	Se connecter avec student@elearn.com / student123	Le tableau de bord etudiant s'affiche avec statistiques et progression.
3	Consultation cours	Ouvrir Courses puis choisir Web Development Fundamentals	Le catalogue et la page lecon affichent le contenu pedagogique.
4	Suivi progression	Naviguer dans les lecons du cours	La progression est sauvegardee dans la table progress.
5	Quiz	Ouvrir le quiz du cours, repondre aux questions	Le score est calcule, affiche et sauvegarde dans quiz_results.
6	Examen final	Ouvrir Final Exam, generer 20 questions, soumettre les reponses	Les questions sont variees, corrigees, et le resultat est sauvegarde dans exam_results.
7	Certificat	Obtenir au moins 70% avec progression complete	Un certificat est cree avec un identifiant unique et peut etre telecharge.
8	Notes personnelles	Ouvrir Notes, ajouter une note et enregistrer	La note apparait dans la page et reste liee a l'utilisateur connecte.
9	Classement	Ouvrir Leaderboard	Le classement affiche les meilleurs apprenants selon les resultats.
10	Administration	Se connecter avec admin@elearn.com / admin123 puis ouvrir /admin	L'administrateur gere les cours, lecons et utilisateurs.
11	Generation IA admin	Ouvrir Admin > AI Generate et saisir un sujet	Le site genere un cours structure ou utilise un fallback si Gemini est indisponible.
12	Securite roles	Tenter /admin avec un compte etudiant	L'accès admin est refuse et l'utilisateur est redirige.

Annexe J - Extraits de app.py

Les extraits suivants proviendrait du fichier `app.py`, qui contient la logique principale du site : configuration Flask, base SQLite, authentification, routes, APIs IA, quiz, examens et certificats.

Extrait I.1 - Initialisation Flask et base de donnees

```
app = Flask(__name__)
app.secret_key = "elearn_demo_secret_key_not_for_prod"
EXAM_CACHE = {}

DATABASE = os.path.join(os.path.dirname(__file__), "elearn.db")

GEMINI_API_KEY = "AIzaSyDxEkl4iuoqg8HlfeJL_0B25-rhOjY_EIU"

# ████████████████████████████████████████████████████████████████
# COURSES & LESSONS DATA
# ████████████████████████████████████████████████████████████████

STATIC_COURSES = [
    {"id": "course1", "title": "Web Development Fundamentals", "desc": "Start your web journey with HTML, CSS, and JavaScript. Build real-world projects."},
    {
        "lessons": [
            {"id": "l1", "title": "Introduction to HTML", "desc": "HTML (HyperText Markup Language) is the foundation of every web page. In this lesson, you will learn the basic syntax and elements of HTML."},
            {"id": "l2", "title": "CSS Styling Basics", "desc": "CSS (Cascading Style Sheets) controls the visual presentation of your HTML documents. You will explore how to style elements like colors, fonts, and layouts."},
            {"id": "l3", "title": "JavaScript Basics", "desc": "JavaScript adds interactivity and logic to your web pages.\n\n• Variables: Learn how to declare and use variables to store data in your programs."}
        ]
    },
    {"id": "course2", "title": "Python for Data Science", "desc": "Learn Python programming and apply it to real-world data analysis and machine learning tasks."},
    {
        "lessons": [
            {"id": "l1", "title": "Python Fundamentals", "desc": "Python is a high-level, interpreted programming language known for its simplicity and readability. This lesson covers the basics of Python syntax and data types."},
            {"id": "l2", "title": "NumPy & Pandas", "desc": "NumPy and Pandas are the core libraries for data science in Python.\n\nNumPy: Used for numerical computations and working with arrays. Pandas: Used for data manipulation and analysis."}
        ]
    }
]
```

Extrait I.2 - Tables principales SQLite

```
CREATE TABLE IF NOT EXISTS users (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  email TEXT UNIQUE NOT NULL,
  password TEXT NOT NULL,
  name TEXT NOT NULL,
  role TEXT NOT NULL DEFAULT 'student'
);
CREATE TABLE IF NOT EXISTS notes (
  id TEXT PRIMARY KEY,
  user_id INTEGER NOT NULL,
  title TEXT,
  content TEXT,
  color TEXT DEFAULT 'cyan',
  date TEXT
);
CREATE TABLE IF NOT EXISTS courses (
  id TEXT PRIMARY KEY,
  title TEXT NOT NULL,
  description TEXT,
  emoji TEXT DEFAULT 'book',
  color TEXT DEFAULT '#1a2a4a',
  level TEXT DEFAULT 'beginner',
  hours INTEGER DEFAULT 0,
  category TEXT DEFAULT 'general',
  created_by INTEGER,
  created_at TEXT
);
CREATE TABLE IF NOT EXISTS lessons (
  id TEXT PRIMARY KEY,
  course_id TEXT NOT NULL,
  title TEXT NOT NULL,
  content TEXT,
  lesson_order INTEGER DEFAULT 0
);
CREATE TABLE IF NOT EXISTS progress (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  user_id INTEGER NOT NULL,
  course_id TEXT NOT NULL,
  lesson_id TEXT NOT NULL,
  visited_at TEXT NOT NULL,
  UNIQUE(user_id, lesson_id)
);
CREATE TABLE IF NOT EXISTS quiz_results (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  user_id INTEGER NOT NULL,
  course_id TEXT NOT NULL,
  score INTEGER NOT NULL,
  total INTEGER NOT NULL,
  percentage REAL NOT NULL,
  taken_at TEXT NOT NULL
);
CREATE TABLE IF NOT EXISTS completions (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  user_id INTEGER NOT NULL,
  course_id TEXT NOT NULL,
  quiz_score REAL NOT NULL,
```

```
        certificate_id TEXT UNIQUE NOT NULL,
        completed_at TEXT NOT NULL,
        UNIQUE(user_id, course_id)
    );
CREATE TABLE IF NOT EXISTS exam_results (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    user_id INTEGER,
    course_id TEXT,
    score INTEGER,
    total INTEGER,
    percentage REAL,
    created_at TEXT
);
CREATE TABLE IF NOT EXISTS exam_completions (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    user_id INTEGER,
    course_id TEXT,
    percentage REAL,
    certificate_id TEXT UNIQUE,
    created_at TEXT,
    UNIQUE(user_id, course_id)
);
""")
```

Extrait I.3 - Controle des sessions et roles

```
def login_required(f):
    @wraps(f)
    def decorated(*args, **kwargs):
        if "user_id" not in session:
            flash("Please sign in to continue.", "info")
            return redirect(url_for("login"))
        return f(*args, **kwargs)
    return decorated

def admin_required(f):
    @wraps(f)
    def decorated(*args, **kwargs):
        if "user_id" not in session:
            return redirect(url_for("login"))
        if session.get("role") != "admin":
            flash("Admin access required.", "danger")
            return redirect(url_for("dashboard"))
        return f(*args, **kwargs)
    return decorated
```

Extrait I.4 - Route dashboard et statistiques

```
@app.route("/dashboard")
@login_required
def dashboard():
    db = get_db()
    user_id = session["user_id"]
    ensure_certificates_for_user(user_id)
    notes_count = db.execute("SELECT COUNT(*) FROM notes WHERE user_id=?", (user_id,)).fetchone()[0]
    stats = get_user_stats(user_id)
    courses_rows = db.execute("SELECT * FROM courses ORDER BY created_at ASC").fetchall()
    courses = []
    for c in courses_rows:
        lessons_rows = db.execute("SELECT * FROM lessons WHERE course_id=? ORDER BY lesson_order", (c["id"],)).fetchall()
        courses.append({
            "id": c["id"], "title": c["title"], "desc": c["description"],
            "emoji": c["emoji"], "color": c["color"], "level": c["level"],
            "lessons_count": len(lessons_rows), "hours": c["hours"],
            "category": c["category"], "progress": get_course_progress(user_id, c["id"]),
            "lessons": [{"id": l["id"], "title": l["title"], "desc": l["content"]} for l in lessons_rows],
        })
    recently_visited = db.execute(
        """
        SELECT p.course_id, p.lesson_id, p.visited_at, c.title AS course_title,
               c.emoji AS course_emoji, l.title AS lesson_title
        FROM progress p
        JOIN courses c ON c.id = p.course_id
        JOIN lessons l ON l.id = p.lesson_id
        WHERE p.user_id=?
        ORDER BY p.visited_at DESC
        LIMIT 3
        """,
        (user_id,),
    ).fetchall()
    recommended = [c for c in courses if c["progress"] == 0][:3]
    continue_course = next((c for c in courses if c["id"] == recently_visited[0]["course_id"]), None) if recently_visited else None
    if continue_course is None and courses:
        continue_course = courses[0]
    return render_template(
        "dashboard.html", user=current_user(), notes_count=notes_count, courses=courses[:6],
        stats=stats, real_lessons_completed=stats["total_lessons_completed"],
        real_quizzes_passed=stats["quizzes_passed"], real_certificates=stats["certificates"],
        recently_visited=recently_visited, recommended=recommended, continue_course=continue_course,
    )
```

Extrait I.5 - Generation et correction de l'examen

```
import os
import sqlite3
import json
import uuid
from datetime import datetime
from functools import wraps
import requests
from io import BytesIO
from flask import (
    Flask, render_template, request, redirect, url_for,
    session, flash, jsonify, g, send_file
)

app = Flask(__name__)
app.secret_key = "elearn_demo_secret_key_not_for_prod"
EXAM_CACHE = {}

DATABASE = os.path.join(os.path.dirname(__file__), "elearn.db")

GEMINI_API_KEY = "AIzaSyDxEkl4iuoqg8HlfeJL_0B25-rh0JY_EIU"

# #####
# COURSES & LESSONS DATA
# #####
STATIC_COURSES = [
    {"id": "course1", "title": "Web Development Fundamentals", "desc": "Start your web journey with HTML, CSS, and JavaScript. Build real-world projects."},
    {
        "lessons": [
            {"id": "l1", "title": "Introduction to HTML", "desc": "HTML (HyperText Markup Language) is the foundation of every web page. It defines the structure and content of documents displayed as web pages."},
            {"id": "l2", "title": "CSS Styling Basics", "desc": "CSS (Cascading Style Sheets) controls the visual presentation of your HTML documents, including layout, colors, fonts, and animations."},
            {"id": "l3", "title": "JavaScript Basics", "desc": "JavaScript adds interactivity and logic to your web pages. Learn variables, functions, loops, and DOM manipulation."}
        ]
    },
    {"id": "course2", "title": "Python for Data Science", "desc": "Learn Python programming and apply it to real-world data analysis and machine learning tasks."},
    {
        "lessons": [
            {"id": "l1", "title": "Python Fundamentals", "desc": "Python is a high-level, interpreted programming language known for its simplicity and readability. This lesson covers basic syntax, data types, and control flow."}
        ]
    }
]
```